Optimus-Cost-Agent: Strategic Roadmap

Phase 2 and Phase 3 distributed architectures (v11)

Architected by: Vibhanshu Agarwal. v11 is the capability and evidence correction of v10 (2026-10-15), prepared by Claude under the operator's documentation-correction request and reviewed at the plan's checkpoints. It keeps v10's intended architecture and changes no approved requirement. It states, for each phase, what is implemented, what is an approved requirement and what is proposed, and it replaces unconditional performance claims with scoped design aims and the measurements that would test them. Predecessor: Optimus-Architecture-Roadmap-v10.pdf (archived unchanged).

1. Executive Summary

Phase 1 of the Optimus-Cost-Agent establishes local request-level cost attribution and automated harness engineering as a Python ACP server behind a loopback Gateway. The subsequent roadmap targets native platform isolation and team federation. Phase 3 evolves the architecture into a governed Agentic Mesh, using the Model Context Protocol (MCP) as the enterprise-wide connective tissue.

Phase	Status at 2026-10-15	Basis
Phase 1: Python local agent and Optimus Gateway	Components implemented and offline-verified; several capabilities remain inactive or deferred (HLD v2.20 §15); the Phase 1 release gate is not established by this correction (§4)	Source `91fd64a`; sealed offline suites at cf9bb9d; operator acceptance of the offline delivery 2026-10-08
Phase 2: Rust local multi-agent swarm	Proposed design. No Rust source, Cargo manifest or tokio dependency exists in the repository	HLD §10.C names the trigger; no plan, issue or schedule exists
Phase 3: distributed enterprise Agentic Mesh	Proposed design	HLD §10.C names the trigger; no plan, issue or schedule exists

This correction schedules or delivers no phase. The HLD's Phase 1 release gate (agent process holds only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`) remains the precondition for any later phase; whether it is satisfied is not established by this correction (§4).

2. Phase 2: Rust Local Multi-Agent Swarm (proposed)

Phase 2 proposes decoupling the runtime from Python execution boundaries by moving core supervision to a native Rust background daemon built on the actor model (tokio). The design emphasises runtime safety, predictable idle behaviour and lifecycle isolation over raw memory savings. Each aim below is a design aim with a verification need, not a delivered property.

- **Actor supervision loop.** Core agent routing, verification and validation states run as lightweight asynchronous tasks under an explicit supervision layout that handles unexpected worker closures. Verification need: supervisor restart and backpressure behaviour under induced worker failure.
- **OS process isolation for external tools.** External tools, web scrapers and MCP servers run outside the daemon boundary in separate, lower-privilege operating-system processes. The aim is to limit what a compromised tool process can reach; the strength of that limit depends on the OS sandboxing mechanism actually used and must be verified per platform. Process separation by itself does not completely shield file hierarchies or the host, and this roadmap claims no such guarantee.
- **Stable idle footprint.** The aim is a supervisor without runtime garbage-collection pauses and with low, predictable wake latency when an IDE protocol notification arrives. Millisecond-scale hook response is a hypothesis to measure on fixed hardware, not a guarantee. The garbage-collection aim applies to the Rust supervisor process only; any Python worker, test process, build or cache it still runs keeps its own runtime characteristics.

2.1 What a Rust supervisor would and would not change

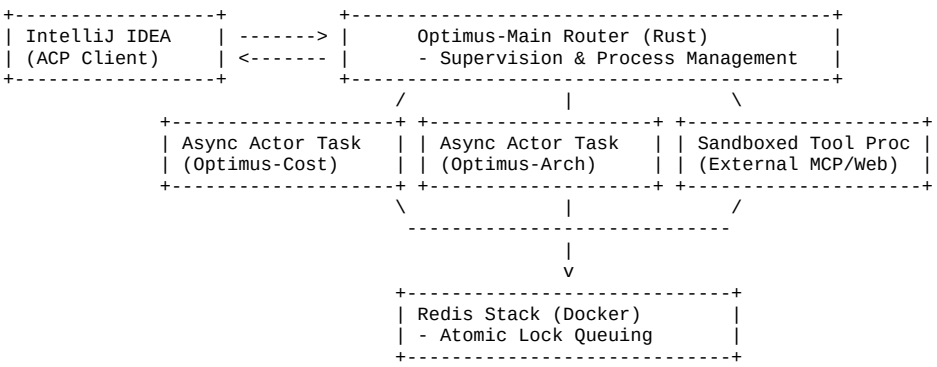
A Rust supervisor could change the supervisor process's own memory, idle CPU, startup and cancellation behaviour. It does not by itself reduce Python test-worker memory, coverage-measurement cost, build or cache I/O, or NVMe contention from those workloads, because those costs belong to the worker processes and the work they do. Any claim about the whole workload must measure the whole process tree and separate the supervisor from its workers.

2.2 Future benchmark dimensions (planned, UNRUN)

No benchmark is implemented or commissioned by this document, and no speedup, cost comparison or acceptance threshold is asserted. When Phase 2 is scheduled, its measurements should fix the source revision, environment, hardware and workload, and report, separately for the supervisor and for workers, build and cache:

Dimension	Measurement	Separation
Memory	Process-tree resident set (RSS) over time	Supervisor vs each worker class
Idle cost	Idle CPU and wake latency with no requests	Supervisor only, then with idle workers attached
Startup and latency	Cold start, first notification to first action, steady-state per-request latency	Supervisor path vs worker path
Cancellation and teardown	Time to cancel in-flight work and to tear down the process tree; orphan count	Per process class
I/O	Bytes and operations for build, cache and test traffic	Workers, build and cache separately from the supervisor

Phase 2 high-level design (proposed)



3. Phase 3: Distributed Enterprise Agentic Mesh (proposed)

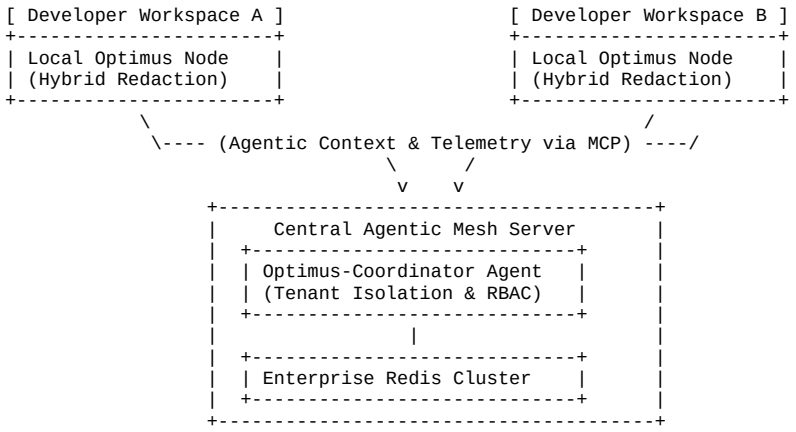
Phase 3 proposes moving the platform from a local productivity utility to an interconnected Agentic Mesh. By treating architectural governance as a distributed concern, it would bridge scattered local desktop environments with a centralised cluster orchestrator. Nothing in this section is implemented; the Phase 1 Gateway is loopback-only and adds no MCP endpoint (HLD §10.A, LLD §0).

A. Agentic context and enterprise governance plane

To operate within strict enterprise security regulations and enforce cross-team standards, the proposed central cluster incorporates multi-tier defence and context negotiation nodes:

- **Cross-swarm MCP negotiation.** Distributed Optimus daemons would use MCP not only for local tool access but to negotiate architectural state, dependencies and API contracts across business units. MCP carries context and tool access; the negotiation layer would enforce authentication, policy boundaries, schema versioning and tenant isolation.
- **Hybrid secrets redaction.** Payloads leaving a workstation would undergo deterministic regex template scanning, high-entropy token analysis and validation against locally configured secret registries or key allowlists. Phase 1's local secret scanning and redaction (HLD §13, LLD §12) are the implemented precursors; a network-boundary redaction service is not.
- **Telemetry privacy boundaries.** Fine-grained developer workspace opt-in constraints would restrict metadata transfer to high-level usage attribution records and drop code strings. Phase 1's OTel export already redacts secrets before export (HLD §11A); the opt-in policy plane is proposed.
- **Multi-tenant workspace partitioning.** Workspace data isolation across a shared execution pool, with roles and permissions mapped cleanly. Proposed.

Phase 3 high-level design (proposed)



4. Classification and custody

Statement in this roadmap	Classification
Phase 1 architecture: loopback Gateway, typed tools, usage ledger, guardrails, bounded loops, skills, Context Engine	Implemented components; offline-verified at cf9bb9d and unit-tested at the pin, with the inactive and deferred items stated in HLD v2.20 §15 and Test Strategy v1.9 §16 (Context Engine production attachment and trusted-snapshot enforcement inactive; live and real-client qualification UNRUN)
Phase 1 release gate (HLD §10.C: mandatory before any later phase)	Approved requirement whose definition exists; its satisfaction is not established by this correction: the live credential-scope, route and real-client qualifications it depends on remain UNRUN (Test Strategy v1.9 §16; issue #237), and no acceptance record names it as passed. This table is not a Phase 1 operational-release approval and grants no permission to start Phase 2 or Phase 3
Phase 2 and Phase 3 triggers (HLD §10.C)	Approved requirement: Phase 1 is mandatory before any later phase
Rust supervisor, actor model, sandboxed external-tool processes	Proposed design; no implementation, plan or issue
Millisecond hooks, garbage-collection elimination, complete shielding of file hierarchies (v10 wording)	Replaced by the scoped aims and verification needs in §2
Benchmark dimensions in §2.2	Planned measurement design; UNRUN; no thresholds or comparisons asserted
Phase 3 negotiation, redaction, telemetry and partitioning planes	Proposed design; Phase 1 precursors named where they exist

GitHub Issues are the work register (ADR-022). No issue exists for Phase 2 or Phase 3 at the inspected source; scheduling either phase requires its own issue, plan and operator decision. This document carries scope and intent only.